Documentation Technique — ATLANTIS (AvaloniaApp)

Version: 1.0
Date: 20/04/2026

1. Objectif

Cette documentation décrit l'architecture technique de l'application desktop ATLANTIS, son organisation en couches (UI/MVVM/Data), ses flux principaux, son modèle de données et les points d'attention pour la maintenance.

2. Stack technique

- Framework UI: Avalonia 11.3.9
- Runtime: .NET 9 (`net9.0`)
- Pattern: MVVM
- ORM: Entity Framework Core 9 (`Npgsql.EntityFrameworkCore.PostgreSQL`)
- Base de données: PostgreSQL
- Authentification: Vérification mot de passe via BCrypt (`BCrypt.Net-Next`)
- Toolkit MVVM: `CommunityToolkit.Mvvm`

Références projet:

- `AvaloniaApp.csproj`
- `Program.cs`
- `App.axaml`
- `App.axaml.cs`

3. Organisation du code

3.1 Arborescence principale

- `Data/` : accès données EF Core (`AppDbContext`)
- `Models/` : entités métier mappées aux tables PostgreSQL
- `ViewModels/` : logique applicative, commandes, états UI
- `Views/` : vues Avalonia (`.axaml`) et code-behind des dialogues
- `Services/` : services transverses (session locale)
- `Assets/` : ressources visuelles
- `migrations copy/` et `seeders/` : scripts historiques Laravel conservés en référence

3.2 Démarrage application

1. `Program.Main()` initialise Avalonia et démarre en mode Desktop.
2. `App.OnFrameworkInitializationCompleted()` configure l'application.
3. La fenêtre initiale est `LoginWindow` avec `LoginViewModel` .

Points importants:

- La validation DataAnnotations Avalonia est explicitement désactivée pour éviter les doublons avec CommunityToolkit.
- Le code contient un test d'initialisation DB (sans `EnsureCreated` actif).

4. Architecture MVVM

4.1 Principes

- Les `ViewModel` exposent des propriétés observables (`[ObservableProperty]`) et des commandes (`[RelayCommand]`).
- Les vues sont liées via DataBinding Avalonia.
- La navigation est gérée au niveau de `MainWindowViewModel` via la propriété `CurrentPage` .

4.2 Navigation principale

`MainWindowViewModel` expose les commandes:

- `ShowDashboard`
- `ShowUsers`
- `ShowRoles`
- `ShowCompanyInfo`
- `ShowStatistiques`
- `ShowPersonnel`
- `ShowProfil`
- `Logout`
- `ReloadApplication`

`MainWindow.axaml` déclare les DataTemplates qui mappent chaque ViewModel vers sa vue correspondante.

5. Authentification et session

5.1 Flux de connexion

Implémenté dans `LoginViewModel` :

1. Validation locale des champs email/mot de passe.
2. Chargement utilisateur depuis `users` via `AppDbContext` .
3. Vérification BCrypt du mot de passe hashé.
4. Contrôle d'autorisation: seul `RoleId == 1` (Gérant) est accepté.
5. Ouverture de `MainWindow` en cas de succès.

5.2 Persistance session

`SessionService` sérialise un fichier `session.json` dans le dossier d'exécution:

- `SaveSession(email)`
- `LoadSession()` (expiration 7 jours)
- `ClearSession()`

6. Couche données (EF Core)

6.1 Contexte EF

`AppDbContext` déclare les `DbSet` principaux:

- Utilisateurs et sécurité: `Users` , `Roles` , `Permissions` , `PermissionRoles`
- Entreprise: `CompanyInfos` , `Societes`
- Production / affaires: `Affaires` , `Materiels` , `Cdes` , `Ddps` , `DdpCdeStatuts`
- Personnel: `Personnels` , `AffairePersonnels` , `AffairePersonnelTaches` , `PersonnelConges`

Connexion PostgreSQL configurée dans `OnConfiguring` avec retry transient failures (`EnableRetryOnFailure`).

6.2 Conventions de mapping

Les entités utilisent les attributs:

- `[Table("...")]`
- `[Column("...")]`
- `[Key]`
- `[ForeignKey("...")]`

Exemples:

- `User` -> table `users`
- `Role` -> table `roles`
- `Personnel` -> table `personnels`
- `PersonnelConge` -> table `personnel_conges`

6.3 Règles DateTime (important)

Pour les colonnes PostgreSQL `timestamp with time zone` :

- Utiliser `DateTime.UtcNow` pour `created_at` / `updated_at`
- Forcer les dates saisies en UTC via `DateTime.SpecifyKind(..., DateTimeKind.Utc)`

Cette règle est appliquée notamment dans:

- `PersonnelViewModel`
- `AddEditPersonnelDialog`
- `AddPersonnelCongeDialog`

7. Modules métier (implémentation)

7.1 Utilisateurs (`UsersViewModel`)

Fonctionnalités:

- Chargement utilisateurs actifs (`DeletedAt == null`) avec rôle lié
- Recherche locale (nom/prénom)
- Création utilisateur avec validation (email, nom/prénom, téléphone)
- Hash BCrypt du mot de passe à la création
- Sauvegarde des modifications en lot (`SaveAll`)

Comportements clés:

- Normalisation téléphone sur 10 chiffres
- Rollback visuel via rechargement en cas de validation échouée

7.2 Rôles et permissions (`RolesViewModel`)

Fonctionnalités:

- CRUD rôle (création, édition, suppression logique)
- Protection du rôle Gérant contre suppression
- Chargement permissions et affectation au rôle sélectionné
- Sauvegarde par réécriture complète des liens `permission_role`

Comportements clés:

- Si rôle admin/gérant sans permission liée, pré-cochage automatique en UI

7.3 Personnel (`PersonnelViewModel`)

Sous-modules:

- Calendrier: tâches par mois (`AffairePersonnelTaches`) + congés/absences/retards (`PersonnelConges`)
- Liste personnel: ajout, modification, archivage/restauration (via `DeletedAt`)
- Congés/absences/retards: dialogues dédiés et persistance DB

Comportements techniques:

- Borne mensuelle en UTC pour requêtes calendaires
- Détachement navigation (`conge.Personnel = null`) avant `SaveChangesAsync` pour éviter conflit de tracking EF

7.4 Informations entreprise (`CompanyInfoViewModel`)

Fonctionnalités:

- Chargement de l'entité `CompanyInfo` (ID 1)
- Mise à jour des informations
- Sélection logo via FilePicker Avalonia

7.5 Statistiques (`StatistiquesViewModel`)

Fonctionnalités:

- Agrégats métiers pour Affaires, Commandes, DDP, Matériels
- Comptages par statut
- Séries temporelles mensuelles
- Derniers éléments créés

8. Dialogues techniques

8.1 `AddEditPersonnelDialog`

- Mode création ou édition
- Génération automatique matricule (`EMP-XXXX`) à partir du max existant
- Validation nom/prénom obligatoires
- `DateEmbauche` convertie en UTC au moment de l'enregistrement

8.2 `AddPersonnelCongeDialog`

- 3 modes: `conge` , `absence` , `retard`
- Validation des dates (fin >= début)
- Typage événement (`conge_paye` , `conge_maladie` , etc.)

9. Thème et design system

Défini dans `App.axaml` :

- Palette sombre personnalisée (ressources dynamiques)
- Styles globaux boutons, checkbox, DataGrid, cards
- Accent système redéfini

Ressources principales:

- `BackgroundColor`, `SurfaceColor`, `SidebarColor`
- `PrimaryColor`, `PrimaryHoverColor`
- `TextColor`, `BorderColor`, `SuccessColor`, `DangerColor`

10. Exécution et maintenance

10.1 Prérequis

- .NET SDK 9
- Accès réseau à PostgreSQL cible
- Droits SQL adaptés (lecture/écriture sur schéma applicatif)

10.2 Commandes utiles

Depuis la racine projet:

```
dotnet restore
dotnet build
dotnet run
```

10.3 Rechargement applicatif

Le bouton de reload appelle `ReloadApplicationCommand`:

- lance un nouveau `dotnet run`
- ferme l'instance courante
- conserve la session utilisateur

11. Points de vigilance

1. Chaîne de connexion PostgreSQL actuellement codée en dur dans `AppDbContext`.
2. Politique d'authentification liée à `RoleId == 1` (couplage fort au modèle de données).
3. Les scripts `migrations copy/` et `seeders/` sont de provenance Laravel: ils servent de référence mais ne sont pas exécutés par EF Core.
4. Peu de tests automatisés observés dans le dépôt (approche majoritairement manuelle).

12. Recommandations d'évolution

1. Externaliser la configuration DB (appsettings, variables d'environnement, secret store).
2. Introduire une couche de services métier entre ViewModels et EF pour réduire le couplage.
3. Ajouter une gestion centralisée des erreurs (journalisation structurée).
4. Mettre en place des tests unitaires ViewModel et tests d'intégration DB (environnement de test).
5. Formaliser une stratégie de migrations côté .NET si l'application devient source de vérité du schéma.

13. Fichiers de référence (lecture rapide)

- `AvaloniaApp.csproj`
- `Program.cs`
- `App.axaml`
- `App.axaml.cs`
- `Data/AppDbContext.cs`
- `ViewModels/LoginViewModel.cs`

- `ViewModels/MainWindowViewModel.cs`
- `ViewModels/UsersViewModel.cs`
- `ViewModels/RolesViewModel.cs`
- `ViewModels/PersonnelViewModel.cs`
- `Views/MainWindow.axaml`
- `Views/LoginWindow.axaml`